

Recurso de Profundización – Semana 5

“Del DOM dinámico a las interfaces accesibles e inteligentes”

Objetivo general del recurso

Aplicar los principios de manipulación del DOM, validación dinámica, consumo de datos con Fetch API y accesibilidad con atributos ARIA para desarrollar una interfaz web interactiva, persistente y accesible, conforme con las buenas prácticas de la programación moderna en JavaScript.

Ejercicio 1 – Manipulación del DOM y eventos

Objetivo:

Comprender cómo crear, modificar y eliminar elementos del DOM mediante eventos del usuario.

Estructura base:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejercicio 1 - DOM y Eventos</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 1rem; }
    .tarea { padding: 0.4rem; margin: 0.3rem 0; background: #f1f5f9; border-radius: 0.5rem; }
    .tarea:hover { background: #e2e8f0; cursor: pointer; }
  </style>
</head>
<body>
  <h1>Lista dinámica de tareas</h1>
  <input id="entrada" placeholder="Escribe una tarea..." />
  <button id="agregar">Agregar</button>
  <ul id="lista"></ul>

  <script>
    const entrada = document.querySelector("#entrada");
    const lista = document.querySelector("#lista");

    // Agregar una tarea al presionar el botón
    document.querySelector("#agregar").addEventListener("click", () => {
      const texto = entrada.value.trim();
      if (!texto) return;
      const li = document.createElement("li");
      li.textContent = texto;
      li.className = "tarea";
      lista.appendChild(li);
      entrada.value = "";
    });

    // Eliminar tarea al hacer clic sobre ella
    lista.addEventListener("click", (e) => {
      if (e.target.tagName === "LI") e.target.remove();
    });
  </script>
</body>
</html>
```

Resultado: Cada clic en “Agregar” crea una nueva tarea. Al hacer clic en una tarea, esta se elimina del DOM dinámicamente.

Ejercicio 2 – Validación dinámica con DOM API + Storage

Objetivo:

Validar campos en tiempo real y almacenar los datos válidos de forma persistente en `localStorage` o temporal en `sessionStorage`.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejercicio 2 - Validación dinámica</title>
  <style>
    input { display:block; margin-bottom:0.5rem; padding:0.4rem; width:250px; }
    small { color:red; min-height:1.2em; display:block; }
    input.ok { border:2px solid green; }
    input.error { border:2px solid red; }
  </style>
</head>
<body>
  <h1>Formulario con validación y almacenamiento</h1>
  <form id="formulario">
    <label>Nombre:</label>
    <input id="nombre" placeholder="Tu nombre completo" />
    <small id="errorNombre"></small>

    <label>Correo:</label>
    <input id="correo" type="email" placeholder="tu@correo.com" />
    <small id="errorCorreo"></small>

    <button>Guardar</button>
  </form>

  <script>
    const nombre = document.querySelector("#nombre");
    const correo = document.querySelector("#correo");

    // Cargar valores guardados
    nombre.value = localStorage.getItem("nombre") || "";
    correo.value = sessionStorage.getItem("correo") || "";

    // Función de validación
    function validarCampo(campo, regex, errorElem, clave, storage) {
      const valido = regex.test(campo.value.trim());
      campo.className = valido ? "ok" : "error";
      campo.setAttribute("aria-invalid", !valido);
      errorElem.textContent = valido ? "" : "Formato inválido.";
      if (valido) storage.setItem(clave, campo.value);
    }

    nombre.addEventListener("input", () =>
      validarCampo(nombre, /^[A-Za-zÁÉÍÓÚáéíóúñÑ ]{3,}$/ , errorNombre, "nombre",
        localStorage));

    correo.addEventListener("input", () =>
      validarCampo(correo, /^[^@\s]+@[^@\s]+\.[^@\s]+$/ , errorCorreo, "correo",
        sessionStorage));
  </script>
</body>
</html>
```

Comportamiento:

- Los datos válidos se guardan automáticamente.

- Si se recarga la página, el nombre persiste (localStorage) y el correo solo mientras dure la sesión.

Ejercicio 3 – Fetch API + IndexedDB

Objetivo:

Consumir una API remota, mostrar los resultados y almacenarlos en una base local (IndexedDB) para acceso offline.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejercicio 3 - Fetch + IndexedDB</title>
  <style>
    body { font-family: sans-serif; margin: 1rem; }
    button { margin-bottom: 1rem; padding: .5rem; }
    li { background:#f9fafb; margin:.3rem 0; padding:.4rem; border-radius:.3rem; }
  </style>
</head>
<body>
  <h1>Usuarios (API → IndexedDB)</h1>
  <button id="btnCargar">Cargar usuarios</button>
  <ul id="usuarios"></ul>

  <script>
    const DB_NAME = "UsuariosDB";
    let db;

    const req = indexedDB.open(DB_NAME, 1);
    req.onupgradeneeded = e => e.target.result.createObjectStore("usuarios", { keyPath:
"id" });
    req.onsuccess = e => db = e.target.result;

    function guardar(users) {
      const tx = db.transaction("usuarios", "readwrite");
      const store = tx.objectStore("usuarios");
      users.forEach(u => store.put(u));
    }

    function leer(callback) {
      const tx = db.transaction("usuarios", "readonly");
      const store = tx.objectStore("usuarios");
      const req = store.getAll();
      req.onsuccess = () => callback(req.result);
    }

    async function cargarDatos() {
      const lista = document.querySelector("#usuarios");
      lista.innerHTML = "<li>Cargando...</li>";
      try {
        const res = await fetch("https://jsonplaceholder.typicode.com/users");
        const data = await res.json();
        guardar(data);
        lista.innerHTML = data.map(u => `<li>${u.name} (${u.email})</li>`).join("");
      } catch {
        leer(users => lista.innerHTML = users.map(u => `<li>${u.name}
(${u.email})</li>`).join(""));
      }
    }

    document.querySelector("#btnCargar").addEventListener("click", cargarDatos);
  </script>
</body>
</html>
```

Resultado:

- Si hay conexión, los datos se descargan y guardan localmente.
- Si no hay red, la app muestra los datos almacenados en IndexedDB.

Ejercicio 4 – Accesibilidad con ARIA y validación

Objetivo:

Aplicar atributos ARIA a una validación dinámica para hacerla comprensible para lectores de pantalla.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejercicio 4 - Accesibilidad ARIA</title>
  <style>
    input { display:block; margin-bottom:.5rem; padding:.4rem; }
    small { color:red; min-height:1.2em; display:block; }
  </style>
</head>
<body>
  <h1>Formulario accesible con ARIA</h1>
  <form id="registro">
    <label for="usuario">Usuario:</label>
    <input id="usuario" aria-describedby="errUsuario" aria-invalid="false" />
    <small id="errUsuario" aria-live="assertive"></small>

    <label for="clave">Contraseña:</label>
    <input id="clave" type="password" aria-describedby="errClave" aria-invalid="false" />
    <small id="errClave" aria-live="assertive"></small>

    <button>Enviar</button>
  </form>

  <script>
    const usuario = document.querySelector("#usuario");
    const clave = document.querySelector("#clave");

    usuario.addEventListener("input", () => {
      const valido = usuario.value.length >= 3;
      usuario.setAttribute("aria-invalid", !valido);
      errUsuario.textContent = valido ? "" : "Debe tener al menos 3 caracteres";
    });

    clave.addEventListener("input", () => {
      const valido = /^(?=.*\d)(?=.*[A-Z])(?=.*[a-z]).{8,}$/ .test(clave.value);
      clave.setAttribute("aria-invalid", !valido);
      errClave.textContent = valido ? "" : "Debe tener mayúscula, minúscula y número";
    });
  </script>
</body>
</html>
```

Comportamiento:

- Los errores se anuncian por voz (lectores de pantalla).
- Los estados `aria-invalid` cambian dinámicamente.
- Cumple con los principios **POUR** de accesibilidad.

Ejercicio 5 – Proyecto integrador: “Agenda inteligente accesible”

Objetivo general:

Combinar todos los temas (DOM, validación, Fetch, almacenamiento y ARIA) para construir una mini aplicación completa.

Estructura y funcionalidad:

- Permite agregar contactos con nombre y correo.
- Valida los campos dinámicamente.
- Guarda los contactos en IndexedDB.
- Permite cargar contactos externos desde una API.
- Utiliza atributos ARIA para accesibilidad.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Agenda Inteligente Accesible</title>
  <style>
    body { font-family:sans-serif; padding:1rem; }
    .ok { border:2px solid green; }
    .error { border:2px solid red; }
    small { color:red; min-height:1.2em; display:block; }
    ul { list-style:none; padding:0; }
    li { padding:.4rem; background:#f1f5f9; border-radius:.4rem; margin:.3rem 0; }
  </style>
</head>
<body>
  <h1>Agenda Inteligente Accesible</h1>
  <form id="agenda">
    <label>Nombre:</label>
    <input id="nombre" aria-describedby="errNombre" />
    <small id="errNombre" aria-live="assertive"></small>

    <label>Correo:</label>
    <input id="correo" aria-describedby="errCorreo" />
    <small id="errCorreo" aria-live="assertive"></small>

    <button>Guardar contacto</button>
  </form>

  <h2>Contactos guardados</h2>
  <button id="importar">Importar desde API</button>
  <ul id="contactos" aria-live="polite"></ul>

  <script>
    // === IndexedDB ===
    const req = indexedDB.open("AgendaDB", 1);
    req.onupgradeneeded = e => e.target.result.createObjectStore("contactos", { keyPath:
"correo" });
    req.onsuccess = e => db = e.target.result;
    let db;

    function guardar(contacto) {
      const tx = db.transaction("contactos", "readwrite");
      tx.objectStore("contactos").put(contacto);
      listar();
    }

    function listar() {
      const tx = db.transaction("contactos", "readonly");
      const store = tx.objectStore("contactos");
      const req = store.getAll();
      req.onsuccess = () => {
```

```

        contactos.innerHTML = req.result.map(c => `<li>${c.nombre} -
${c.correo}</li>`).join("");
    };
}

// === Validación ===
const nombre = document.querySelector("#nombre");
const correo = document.querySelector("#correo");

function validar() {
    const nValido = /^[A-Za-zÁÉÍÓÚáéíóúñ]{3,}$/.test(nombre.value);
    const cValido = /^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(correo.value);
    nombre.className = nValido ? "ok" : "error";
    correo.className = cValido ? "ok" : "error";
    errNombre.textContent = nValido ? "" : "Nombre inválido";
    errCorreo.textContent = cValido ? "" : "Correo inválido";
    nombre.setAttribute("aria-invalid", !nValido);
    correo.setAttribute("aria-invalid", !cValido);
    return nValido && cValido;
}

agenda.addEventListener("submit", e => {
    e.preventDefault();
    if (validar()) guardar({ nombre: nombre.value, correo: correo.value });
});

// === Importar desde API externa ===
document.querySelector("#importar").addEventListener("click", async () => {
    try {
        const res = await fetch("https://jsonplaceholder.typicode.com/users");
        const data = await res.json();
        data.forEach(u => guardar({ nombre: u.name, correo: u.email }));
    } catch {
        alert("Error al importar contactos");
    }
});
</script>
</body>
</html>

```

Logros integrados:

- DOM y eventos (creación y renderizado).
- Validación dinámica y ARIA (aria-invalid, aria-live).
- Consumo de datos con Fetch API.
- Persistencia completa en IndexedDB.
- Interfaz accesible, sin recarga de página.